

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
"Белгородский государственный технологический университет им. В. Г.
Шухова"
(БГТУ им. В.Г. Шухова)

Институт энергетики, информационных технологий и управляющих систем

Кафедра программного обеспечения вычислительной техники
и автоматизированных систем

Лабораторная работа №1
по дисциплине: ТАиФЯ
по теме: Формальные грамматики. Выводы

Выполнил: студент группы ПВ-221
Дорошенко Владимир Юрьевич
Проверил: Рязанов Ю. Д.

Цель работы: изучить основные понятия теории формальных языков и грамматик.

Задание

1. Написать программу, выполняющую левый вывод в заданной КС-грамматике.

Исходные данные: КС-грамматика.

Выполнение: на каждом шаге вывода отображается промежуточная цепочка (на первом шаге она представляет собой начальный нетерминал, на последнем — терминальную цепочку) и правила, которые можно применить на данном шаге. Пользователь выбирает одно из предложенных правил и процесс повторяется.

Результат: терминальная цепочка, последовательность номеров правил, участвовавших в её выводе, дерево вывода в линейной скобочной форме (ЛСФ ДВ).

2. Выполнить левый (правый) вывод терминальной цепочки в заданной грамматике (см. варианты заданий п.1), построить дерево вывода. Определить, существует ли неэквивалентный вывод полученной цепочки и, если существует, представить его деревом вывода.

3. Написать программу, определяющую, можно ли применить заданную последовательность правил при левом выводе цепочки в заданной КС-грамматике.

Исходные данные:

— КС-грамматика;

— последовательность номеров правил.

Результат: “да” или “нет”.

4. Для каждой последовательности правил (см. варианты заданий п.2) определить, можно ли её применить при левом (правом) выводе терминальной цепочки в заданной КС-грамматике, и, если можно, построить дерево вывода.

5. Написать программу, определяющую, можно ли применить заданную последовательность правил при выводе цепочки в заданной КС-грамматике.

6. Для каждой последовательности правил (см. варианты заданий п.2) определить, можно ли её применить при выводе терминальной цепочки в заданной КС-грамматике, и, если можно, построить дерево вывода и записать эквивалентные левый и правый вывод.

Вариант 8

1. КС-грамматика

- | | |
|-------------------------|-----------------------|
| 1. $S \rightarrow aSbA$ | 5. $A \rightarrow aB$ |
| 2. $S \rightarrow aB$ | 6. $A \rightarrow S$ |
| 3. $S \rightarrow A$ | 7. $B \rightarrow b$ |
| 4. $A \rightarrow aAbS$ | 8. $B \rightarrow aA$ |

2. Последовательности правил вывода

- 1, 3, 4, 5, 7, 2, 7, 5, 7
- 1, 1, 2, 7, 5, 7, 3, 2, 7
- 1, 5, 7, 3, 4, 2, 7, 5, 7
- 1, 3, 1, 5, 2, 2, 7, 7, 7

1. Написать программу, выполняющую левый вывод в заданной КС-грамматике.

main.cpp

```
#include "Chain.h"
#include "windows.h"
#include <locale>

int main() {
    SetConsoleCP(CP_UTF8);
    SetConsoleOutputCP(CP_UTF8);
    std::setlocale(LC_ALL, "ru");

    int nNonTerminals;
    std::cout << "Введите количество нетерминалов в КС-грамматике\n";

    std::cin >> nNonTerminals;
    std::vector<std::vector<Rule>> vecRules(nNonTerminals);

    int count = 1;
    for (int i = 0; i < nNonTerminals; ++i) {
        std::cout << "Введите нетерминал, количество правил, которые для него  
существуют, и затем сами правила\n";
        std::string nonTerminal;
        int nRules;
        std::cin >> nonTerminal >> nRules;

        for (int j = 0; j < nRules; ++j) {
            std::string rule;
            std::cin >> rule;
            if (rule == "~")
                rule = "";
            vecRules[i].push_back({ count, nonTerminal, rule });
            count++;
        }
    }

    CFGGrammar grammar(vecRules);
    Chain chain(grammar);
    chain.leftOutput();

    return 0;
}
```

Grammar.h

```
#include "Rule.h"
#include <map>
#include <vector>
#include <regex>
#include "windows.h"

class CFGGrammar {
public:

    std::map<std::string, std::map<int, std::string>> grammarRules;

    CFGGrammar() = default;

    CFGGrammar(std::vector<std::vector<Rule>>& rules) {
        for (auto& rulesOneNonTerminal : rules) {
            std::map<int, std::string> curNonTerminalRules;
            std::string curNonTerminal = rulesOneNonTerminal[0]._left;
            for (auto& rule : rulesOneNonTerminal) {
                curNonTerminalRules.emplace(rule._index, rule._right);
            }
            grammarRules.emplace(curNonTerminal, curNonTerminalRules);
        }
    };

    static void printRulesForNonTerminal(const std::string& nonTerminal, const
std::map<int, std::string>& rules) {
        for (const auto& rule : rules)
            std::cout << rule.first << ". " << nonTerminal << " -> " << rule.second
<< '\n';
    }

    void printRule(std::string& curNonTerminal) {
        std::cout << "Правила для " << curNonTerminal << ":" << '\n';

        printRulesForNonTerminal(curNonTerminal, grammarRules[curNonTerminal]);
    }
};
```

chain.h

```
#include "CFGGrammar.h"

class Chain {
public:

    Chain(CFGGrammar& grammar) {
        _grammar = grammar;
        currentNonTerminal = "S";
        startNonTerminal = "S";
        currentChain = startNonTerminal;
    }

    void leftOutput() {
        std::regex nonTerminalRegex("[A-Z]");
        std::smatch nonTerminalMatch;
        std::vector<int> listRules;
        bool exist = true;

        while (regex_search(currentChain, nonTerminalMatch,
            nonTerminalRegex)) {
            int index = nonTerminalMatch.position();
            currentNonTerminal = currentChain[index];
            _grammar.printRule(currentNonTerminal);
            std::cout << "\nВыберите правило: ";

            int nom_rule;
            std::cin >> nom_rule;

            if (_grammar.grammarRules[currentNonTerminal].find(nom_rule) !=
                _grammar.grammarRules[currentNonTerminal].end()) {
                listRules.push_back(nom_rule);
                std::string available_rule =
                    _grammar.grammarRules[currentNonTerminal][nom_rule];

                if (available_rule == "e") {
                    available_rule = "";
                }

                currentChain = currentChain.substr(0, index) + available_rule +
                    currentChain.substr(index + 1);
                regex_search(currentChain, nonTerminalMatch,
                    nonTerminalRegex);
                std::cout << "\nЦепочка: " << currentChain << '\n';
            }
            else {
                std::cout << "\nНевозможно применить данное правило" << '\n';
                exist = false;
                break;
            }
        }

        if (exist) {
            std::cout << "\nПолученная цепочка терминальная";
            std::cout << "\nПоследовательность правил вывода: ";
            for (int rule : listRules) {
                std::cout << rule << " ";
            }
            std::cout << '\n';

            int ind = 0;
            std::pair<std::string, int> result = getTree(startNonTerminal,
                listRules, ind);
            std::cout << "\nДерево вывода: " << result.first << '\n';
        }
    }
};
```

```

    }
}

private:
    CFGGrammar _grammar;
    std::string currentNonTerminal;
    std::string startNonTerminal;
    std::string currentChain;

    std::pair<std::string, int> getTree(const std::string& tree,
    const std::vector<int>& numbersOfRules, int indexRule) {
        int i = 0;
        std::string updatedTree = tree;
        while (i < tree.length()) {
            if (isupper(tree[i])) {
                indexRule++;

                const std::string& grm = _grammar.grammarRules[std::string(1,
tree[i])][
                    numbersOfRules[indexRule - 1]];

                std::pair<std::string, int> addPoint = getTree(grm, numbersOfRules,
indexRule);

                updatedTree = updatedTree.substr(0, i + 1) + "(" +
                    addPoint.first + ")" + updatedTree.substr(i + 1);

                i += addPoint.first.length() + 3;
            }
            else {
                i++;
            }
        }
        return { updatedTree, indexRule };
    }
};

```

rule.h

```
#include <iostream>

class Rule {
public:
    Rule(int index, std::string& left, std::string& right) {
        _index = index;
        _left = left;
        _right = right;
    }

    int _index;
    std::string _left;
    std::string _right;
};
```


Получившийся вывод:

```
Введите количество нетерминалов в КС-грамматике
3
Введите нетерминал, количество правил, которые для него существуют, и затем сами правила
S
3
aSbA
aB
A
Введите нетерминал, количество правил, которые для него существуют, и затем сами правила
A
3
aAbS
aB
S
Введите нетерминал, количество правил, которые для него существуют, и затем сами правила
B
2
b
aA
```

Правила для S:

1. $S \rightarrow aSbA$
2. $S \rightarrow aB$
3. $S \rightarrow A$

Выберите правило: 1

Цепочка: aSbA

Правила для S:

1. $S \rightarrow aSbA$
2. $S \rightarrow aB$
3. $S \rightarrow A$

Выберите правило: 2

Цепочка: aaBbA

Правила для B:

7. $B \rightarrow b$
8. $B \rightarrow aA$

Выберите правило: 7

Цепочка: aabbA

Правила для A:

4. $A \rightarrow aAbS$
5. $A \rightarrow aB$
6. $A \rightarrow S$

Выберите правило: 5

Цепочка: aabbaB

Правила для B:

7. $B \rightarrow b$
8. $B \rightarrow aA$

Выберите правило: 7

Цепочка: aabbab

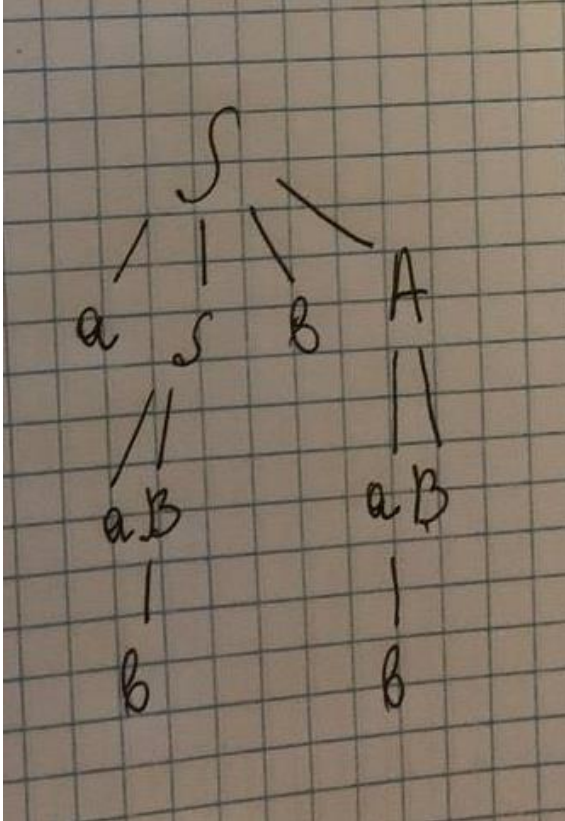
Полученная цепочка терминальная

Последовательность правил вывода: 1 2 7 5 7

Дерево вывода: $S(aS(aB(b))bA)$

Задание №2. Выполнить левый (правый) вывод терминальной цепочки в заданной грамматике (см. варианты заданий п.1), построить дерево вывода. Определить, существует ли неэквивалентный вывод полученной цепочки и, если существует, представить его деревом вывода.

$$S_1 \Rightarrow aS_2bA \Rightarrow aaB_7bA \Rightarrow aabbA_5 \Rightarrow aabbaB_7 \Rightarrow aabbab$$



Неэквивалентный вывод

$$S_3 \Rightarrow A_4 \Rightarrow aA_5bS \Rightarrow aaB_7bS \Rightarrow aabbS_3 \Rightarrow aabbA_5 \Rightarrow aabbaB_7 \Rightarrow aabbab$$

Таким образом, второй вывод: $S \Rightarrow aabbab$

Доказательство неэквивалентности выводов:

В первом выводе первое правило для S , которое было применено, это $S \rightarrow aSbA$, а затем $S \rightarrow aB$. Во втором выводе первое правило для S , которое было применено, это $S \rightarrow AS$, и затем развитие шло через A . Таким образом, сами шаги вывода различаются.

- 1) Первый вывод использует непосредственно замену S на $aSbA$, вывод идет через промежуточные символы B и A .
- 2) Второй вывод сразу заменяет S на A

Это доказывает, что два вывода одной и той же цепочки $aabbab$ в данной грамматике неэквивалентны, так как они используют разные правила и последовательности подстановок.

3. Написать программу, определяющую, можно ли применить заданную последовательность правил при левом выводе цепочки в заданной КС-грамматике.

Новые функции в chain.h

```
void setChainToFirstNonTerminal() {
    currentNonTerminal = "S";
    currentChain = startNonTerminal;
}

bool isSequenceApplicableForLeft(const std::vector<int>& ruleSequence) {
    std::regex nonTerminalRegex("[A-Z]");

    std::smatch nonTerminalMatch;
    bool exist = true;

    for (int rule : ruleSequence) {
        if (!regex_search(currentChain, nonTerminalMatch, nonTerminalRegex)) {
            return false;
        }

        int index = nonTerminalMatch.position();

        currentNonTerminal = currentChain[index];

        if (_grammar.grammarRules.find(currentNonTerminal) ==
            _grammar.grammarRules.end() ||
            _grammar.grammarRules[currentNonTerminal].find(rule) ==
            _grammar.grammarRules[currentNonTerminal].end()) {
            return false;
        }

        std::string availableRule =
            _grammar.grammarRules[currentNonTerminal][rule];

        if (availableRule == "ε") {
            availableRule = "";
        }

        currentChain = currentChain.substr(0, index) + availableRule +
            currentChain.substr(index + 1);

        std::cout << rule << "." << currentNonTerminal << " -> "
            << availableRule << '\n';
        std::cout << currentChain << '\n';
    }

    return exist;
}
```

TAandFL_lab1.cpp

```
#include <iostream>
#include "Chain.h"
#include "windows.h"
#include <locale>

int main() {
    SetConsoleCP(CP_UTF8);
    SetConsoleOutputCP(CP_UTF8);
    std::setlocale(LC_ALL, "ru");

    int nNonTerminals;
    std::cout << "Введите количество нетерминалов в КС-грамматике\n";

    std::cin >> nNonTerminals;
    std::vector<std::vector<Rule>> vecRules(nNonTerminals);

    int count = 1;
    for (int i = 0; i < nNonTerminals; ++i) {
        std::cout << "Введите нетерминал, количество правил, которые для него  
существуют, и затем сами правила\n";
        std::string nonTerminal;
        int nRules;
        std::cin >> nonTerminal >> nRules;

        for (int j = 0; j < nRules; ++j) {
            std::string rule;
            std::cin >> rule;
            if (rule == "~")
                rule = "";
            vecRules[i].push_back({ count, nonTerminal, rule });
            count++;
        }
    }

    int nSequences;
    std::cout << "Введите количество последовательностей для проверки применимости  
для левого вывода\n";

    std::cin >> nSequences;

    CFGGrammar grammar(vecRules);
    Chain chain(grammar);

    for (int i = 0; i < nSequences; ++i) {
        int nRules;
        std::cout << "\nВведите размер последовательности\n";
        std::cin >> nRules;
        std::vector<int> sequence(nRules);
        std::cout << "\nВведите последовательность\n";

        for (auto& ruleIndex : sequence) {
            std::cin >> ruleIndex;
        }

        chain.setChainToFirstNonTerminal();
        if (chain.isSequenceApplicableForLeft(sequence))
            std::cout << "\nПоследовательность применима для левого вывода";
        else
            std::cout << "\nПоследовательность не применима для левого вывода";
    }
    return 0;
}
```

Введите количество нетерминалов в КС-грамматике

3

Введите нетерминал, количество правил, которые для него существуют, и затем сами правила

S

3

aSbA

aB

A

Введите нетерминал, количество правил, которые для него существуют, и затем сами правила

A

3

aAbS

aB

S

Введите нетерминал, количество правил, которые для него существуют, и затем сами правила

B

2

b

aA

Введите количество последовательностей для проверки применимости для левого вывода

4

Введите размер последовательности

9

Введите последовательность

1 3 4 5 7 2 7 5 7

1.S -> aSbA

aSbA

3.S -> A

aAbA

4.A -> aAbS

aaAbSbA

5.A -> aB

aaaBbSbA

7.B -> b

aaabbSbA

2.S -> aB

aaabbaBbA

7.B -> b

aaabbabbA

5.A -> aB

aaabbabbaB

7.B -> b

aaabbabbab

Введите последовательность

1 1 2 7 5 7 3 2 7

1.S → aSbA

aSbA

1.S → aSbA

aaSbAbA

2.S → aB

aaaBbAbA

7.B → b

aaabbAbA

5.A → aB

aaabbaBbA

7.B → b

aaabbabbA

Последовательность не применима для левого вывода

Введите размер последовательности

|

Введите размер последовательности

9

Введите последовательность

1 5 7 3 4 2 7 5 7

1.S → aSbA

aSbA

Последовательность не применима для левого вывода

Введите размер последовательности

|

Введите последовательность

1 3 1 5 2 2 7 7 7

1.S → aSbA

aSbA

3.S → A

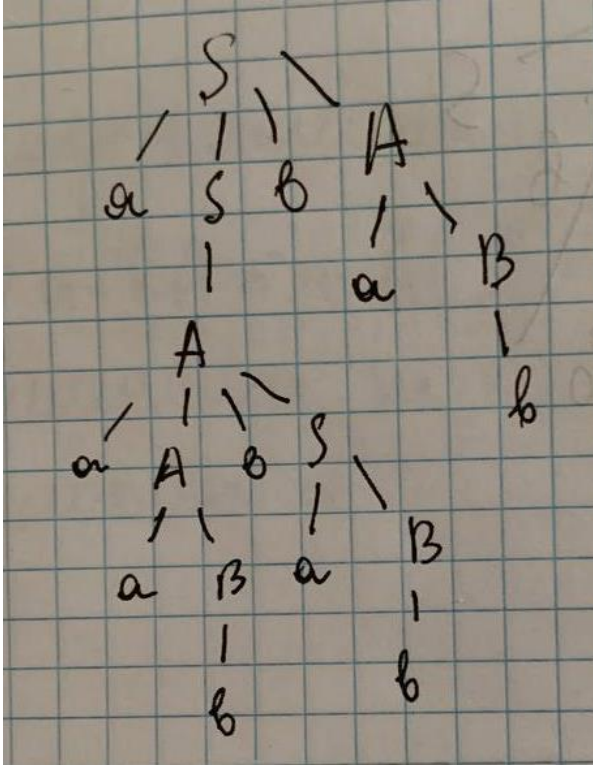
aAbA

Последовательность не применима для левого вывода

Задание №4. Для каждой последовательности правил (см. варианты заданий п.2) определить, можно ли её применить при левом (правом) выводе терминальной цепочки в заданной КС-грамматике, и, если можно, построить дерево вывода.

1. 1, 3, 4, 5, 7, 2, 7, 5, 7

$$S_1 \Rightarrow aS_3bA \Rightarrow aA_4bA \Rightarrow aaA_5bSbA \Rightarrow aaaB_7bSbA \Rightarrow aaabbS_2bA \\ \Rightarrow aaabbaB_7bA \Rightarrow aaabbabbA_5 \Rightarrow aaabbabbaB_7 \Rightarrow aaabbabbab$$



2. 1, 1, 2, 7, 5, 7, 3, 2, 7

$$S \Rightarrow aSbA \Rightarrow aaSbA \Rightarrow aaaBbA \Rightarrow aaabbA \Rightarrow aaabbaB \Rightarrow aaabbab \\ \Rightarrow \text{для того чтобы применить правило 3 нужна } S, \text{ но её нет}$$

5. Написать программу, определяющую, можно ли применить заданную последовательность правил при выводе цепочки в заданной КС-грамматике.

chain.h

```
bool isSequenceApplicable(const std::vector<int>& ruleSequence) {
    std::string chain = startNonTerminal;

    for (int rule : ruleSequence) {

        bool exist = false;

        for (size_t i = 0; i < chain.length(); ++i) {

            if (isupper(chain[i])) {

                currentNonTerminal = std::string(1, chain[i]);

                if (_grammar.grammarRules.find(currentNonTerminal) !=
                    _grammar.grammarRules.end() &&
                    _grammar.grammarRules[currentNonTerminal].find(rule) !=
                    _grammar.grammarRules[currentNonTerminal].end()) {

                    std::string available_rule = _grammar.grammarRules[
                        currentNonTerminal][rule];
                    std::string original_chain = chain;

                    chain.replace(i, 1, available_rule);

                    std::cout << rule << ". " << currentNonTerminal << "-> "
                        << available_rule << '\n';
                    std::cout << chain << '\n';

                    exist = true;
                    break;
                }
            }
        }

        if (!exist) {
            return false;
        }
    }
    return true;
}
```

TAandFL_lab1.cpp

```
#include <iostream>
#include "Chain.h"
#include "windows.h"
#include <locale>

int main() {
    SetConsoleCP(CP_UTF8);
    SetConsoleOutputCP(CP_UTF8);
    setlocale(LC_ALL, "ru");
    int nNonTerminals;
    std::cout << "Введите количество нетерминалов в КС-грамматике\n";
    std::cin >> nNonTerminals;
    std::vector<std::vector<Rule>> vecRules(nNonTerminals);

    int count = 1;
    for (int i = 0; i < nNonTerminals; ++i) {
        std::cout << "Введите нетерминал, количество правил, которые для него  
существуют, и затем сами правила\n";
        std::string nonTerminal;
        int nRules;
        std::cin >> nonTerminal >> nRules;

        for (int j = 0; j < nRules; ++j) {
            std::string rule;
            std::cin >> rule;
            if (rule == "~")
                rule = "";
            vecRules[i].push_back({ count, nonTerminal, rule });
            count++;
        }
    }

    int nSequences;
    std::cout << "Введите количество последовательностей для проверки применимости  
для вывода терминальной цепочки\n";

    std::cin >> nSequences;

    CFGGrammar grammar(vecRules);
    Chain chain(grammar);

    for (int i = 0; i < nSequences; ++i) {
        int nRules;
        std::cout << "\nВведите размер последовательности\n";
        std::cin >> nRules;
        std::vector<int> sequence(nRules);
        std::cout << "\nВведите последовательность\n";

        for (auto& ruleIndex : sequence) {
            std::cin >> ruleIndex;
        }

        chain.setChainToFirstNonTerminal();
        if (chain.isSequenceApplicable(sequence))
            std::cout << "\nПоследовательность применима для вывода терминальной  
цепочки";
        else
            std::cout << "\nПоследовательность не применима для вывода терминальной  
цепочки";
    }
    return 0;
}
```

Введите количество нетерминалов в КС-грамматике

3

Введите нетерминал, количество правил, которые для него существуют, и затем сами правила

S

3

aSbA

aB

A

Введите нетерминал, количество правил, которые для него существуют, и затем сами правила

A

3

aAbS

aB

S

Введите нетерминал, количество правил, которые для него существуют, и затем сами правила

B

2

b

aA

Введите количество последовательностей для проверки применимости для вывода терминальной цепочки

4

Введите размер последовательности

9

Введите последовательность

1 3 4 5 7 2 7 5 7

1. S → aSbA

aSbA

3. S → A

aAbA

4. A → aAbS

aaAbSbA

5. A → aB

aaaBbSbA

7. B → b

aaabbSbA

2. S → aB

aaabbaBbA

7. B → b

aaabbabbA

5. A → aB

aaabbabbabB

7. B → b

aaabbabbab

Последовательность применима для вывода терминальной цепочки

Введите размер последовательности

9

Введите последовательность

1 1 2 7 5 7 3 2 7

1. S→ aSbA

aSbA

1. S→ aSbA

aaSbAbA

2. S→ aB

aaaBbAbA

7. B→ b

aaabbAbA

5. A→ aB

aaabbaBbA

7. B→ b

aaabbabbA

Последовательность не применима для вывода терминальной цепочки

Введите размер последовательности

9

Введите последовательность

1 5 7 3 4 2 7 5 7

1. S→ aSbA

aSbA

5. A→ aB

aSbaB

7. B→ b

aSbab

3. S→ A

aAbab

4. A→ aAbS

aaAbSbab

2. S→ aB

aaAbaBbab

7. B→ b

aaAbabbab

5. A→ aB

aaaBbabbab

7. B→ b

aaabbabbab

Последовательность применима для вывода терминальной цепочки

Введите размер последовательности

9

Введите последовательность

1 3 1 5 2 2 7 7 7

1. $S \rightarrow aSbA$

aSbA

3. $S \rightarrow A$

aAbA

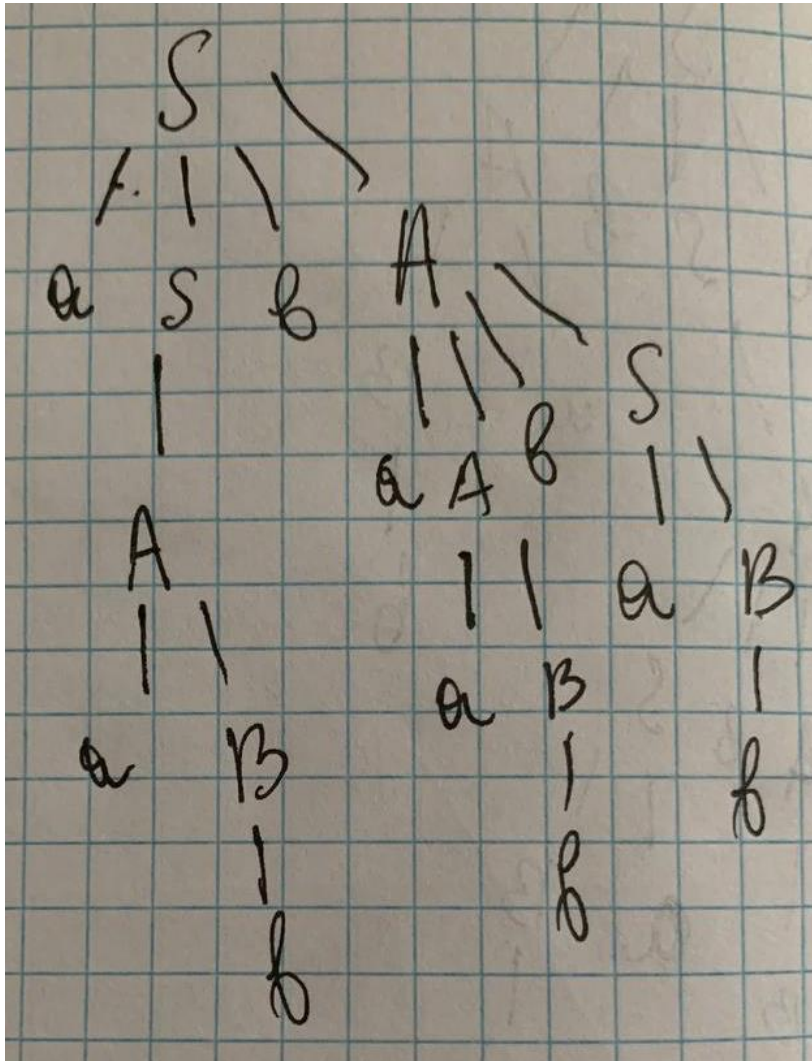
Последовательность не применима для вывода терминальной цепочки

Задание №6. Для каждой последовательности правил (см. варианты заданий п.2) определить, можно ли её применить при выводе терминальной цепочки в заданной КС-грамматике, и, если можно, построить дерево вывода и записать эквивалентные левый и правый вывод.

1. 1, 3, 4, 5, 7, 2, 7, 5, 7

Применю вывод

$$S_1 \Rightarrow aS_3bA \Rightarrow aAbA_4 \Rightarrow aA_5baAbS \Rightarrow aaB_7baAbS \Rightarrow aabbaAbS_2 \\ \Rightarrow aabbaAbaB_7 \Rightarrow aabbaA_5bab \Rightarrow aabbaaB_7bab \Rightarrow aabbaabbab$$



Эквивалентный правый вывод:

$$S_1 \Rightarrow aS_3bA \Rightarrow aAbA_4 \Rightarrow aAbaA_5bS \Rightarrow aAbaaB_7bS \Rightarrow aAbaabbS_2 \\ \Rightarrow aAbaabbaB_7 \Rightarrow aA_5baabbab \Rightarrow aaB_7baabbab \Rightarrow aabbaabbab$$

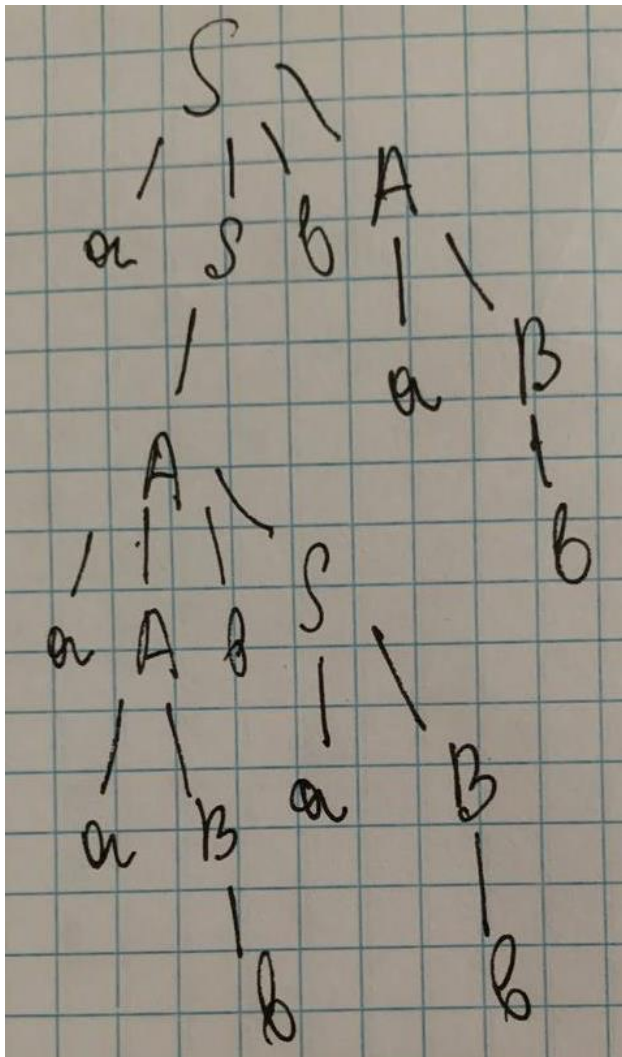
2. 1, 1, 2, 7, 5, 7, 3, 2, 7

$S_1 \Rightarrow aS_1bA \Rightarrow aaS_2bAbA \Rightarrow aaaB_7bAbA \Rightarrow aaabbA_5bA \Rightarrow aaabbaB_7bA \Rightarrow aaabbabbA \Rightarrow$ для того чтобы применить правило 3 нужна S , но её нет

3. 1, 5, 7, 3, 4, 2, 7, 5, 7

Левый вывод

$S_1 \Rightarrow aSbA_5 \Rightarrow aSbaB_7 \Rightarrow aS_3bab \Rightarrow aA_4bab \Rightarrow aaAbS_2bab \Rightarrow aaAbaB_7bab \Rightarrow aaA_5babbab \Rightarrow aaaB_7babbab \Rightarrow aaabbabbab$



Эквивалентный правый вывод

$S_1 \Rightarrow aSbA_5 \Rightarrow aSbaB_7 \Rightarrow aS_3bab \Rightarrow aA_4bab \Rightarrow aaAbS_2bab \Rightarrow aaAbaB_7bab \Rightarrow aaA_5babbab \Rightarrow aaaB_7babbab \Rightarrow aaabbabbab$

4. 1, 3, 1, 5, 2, 2, 7, 7, 7

$S_1 \Rightarrow aS_3bA \Rightarrow aAbA \Rightarrow$ чтобы применить правило 1 нужна S , а её нет